

Звіт

до Лабораторної роботи №11

Collections Framework в Java

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Савченко В. М.

Рік: 2026

1. Мета роботи

Метою лабораторної роботи є отримання практичних навичок використання стандартних колекцій Java.

У роботі потрібно навчитися обирати відповідний тип колекції для конкретної задачі, зберігати об'єкти предметної області, виконувати базові операції над ними та пояснювати, чому було обрано саме такі структури даних.

2. Варіант роботи

Варіант 5 — ігровий профіль

Для обраного варіанта було реалізовано консольну Java-програму для роботи з ігровим профілем гравця.

У програмі використано стандартні колекції Java: List, Set і Map.

3. Опис предметної області

У лабораторній роботі реалізовано просту модель ігрового профілю.

Ігровий профіль містить інформацію про гравця, його предмети та досягнення. Гравець може мати багато ігрових предметів, які зберігаються

у порядку додавання. Також гравець може отримувати досягнення, але однакові досягнення не повинні дублюватися.

Для реалізації цієї задачі були створені такі класи:

- Player — гравець;
- GameItem — ігровий предмет;
- Achievement — досягнення гравця;
- GameProfileService — сервіс для роботи з ігровим профілем;
- Main — демонстраційний клас.

4. Класи предметної області

4.1 Клас Player

Клас Player описує гравця.

Поля класу:

- nickname — нікнейм гравця;
- level — рівень гравця.

Клас містить конструктор, гетери та метод toString().

У конструкторі реалізовано перевірки: нікнейм не може бути порожнім, а рівень гравця має бути додатним числом.

Приклад створення гравця:

```
Player player = new Player("Artem", 15);
```

4.2 Клас GameItem

Клас GameItem описує ігровий предмет.

Поля класу:

- name — назва предмета;
- type — тип предмета;
- power — сила предмета.

Клас містить конструктор, гетери та метод toString().

У конструкторі реалізовано перевірки: назва предмета не може бути порожньою, тип предмета не може бути порожнім, а сила предмета не може бути від'ємною.

Приклад створення предмета:

```
GameItem sword = new GameItem("Сталевий меч", "Зброя", 25);
```

4.3 Клас Achievement

Клас Achievement описує досягнення гравця.

Поля класу:

- title — назва досягнення;
- description — опис досягнення.

Клас містить конструктор, гетери, метод toString(), а також методи equals() і hashCode().

Методи equals() і hashCode() потрібні для коректної роботи колекції Set. У цій роботі два досягнення вважаються однаковими, якщо вони мають однакову назву.

Це дозволяє автоматично не допускати дублювання однакових досягнень у профілі гравця.

5. Клас GameProfileService

Клас GameProfileService є сервісом для роботи з ігровим профілем.

Він зберігає:

```
private final Player player;  
private final List<GameItem> items;  
private final Set<Achievement> achievements;
```

Саме цей клас інкапсулює стандартні колекції Java та надає методи для роботи з ними.

Основні методи сервісу:

- addItem(GameItem item) — додавання предмета;
- findItemByName(String name) — пошук предмета за назвою;
- removeItemByName(String name) — видалення предмета за назвою;
- addAchievement(Achievement achievement) — додавання досягнення;
- hasAchievement(String title) — перевірка наявності досягнення;
- getItems() — отримання списку предметів;
- getAchievements() — отримання набору досягнень;

- getItemCount() — отримання кількості предметів;
- getAchievementCount() — отримання кількості досягнень;
- showProfile() — формування текстового опису профілю.

6. Використані колекції

У роботі використано стандартні колекції Java:

- List<GameItem>;
- Set<Achievement>;
- Map<String, String>.

Ці колекції виконують різні ролі у програмі.

Колекція	Роль у програмі	Причина вибору
ArrayList<GameItem>	Зберігає предмети гравця.	Потрібно зберігати порядок додавання та зручно переглядати елементи.
LinkedHashSet<Achievement>	Зберігає досягнення гравця.	Досягнення мають бути унікальними, а порядок додавання бажано зберегти.
Map<String, String>	Пояснює використані колекції у демонстраційному класі.	Показує асоціативну структуру ключ-значення та обхід через entrySet().

7. Колекція List<GameItem>

Колекція List<GameItem> використовується для зберігання ігрових предметів.

У реалізації використано:

```
private final List<GameItem> items = new ArrayList<>();
```

Ця колекція обрана тому, що предмети потрібно зберігати у порядку додавання, а також список зручно використовувати для послідовного перегляду, пошуку і видалення предмета за назвою.

Метод додавання предмета:

```
public void addItem(GameItem item) {  
    items.add(item);  
}
```

Метод пошуку предмета:

```
public GameItem findItemByName(String name) {  
    for (GameItem item : items) {  
        if (item.getName().equalsIgnoreCase(name)) {  
            return item;  
        }  
    }  
    return null;  
}
```

Метод видалення предмета використовує Iterator, щоб безпечно видалити елемент під час обходу списку.

8. Колекція Set<Achievement>

Колекція Set<Achievement> використовується для зберігання досягнень гравця.

У реалізації використано:

```
private final Set<Achievement> achievements = new LinkedHashSet<>();
```

Ця колекція обрана тому, що досягнення мають бути унікальними. Set автоматично контролює унікальність, а LinkedHashSet додатково зберігає порядок додавання.

Додавання досягнення:

```
public boolean addAchievement(Achievement achievement) {  
    return achievements.add(achievement);  
}
```

Метод add() повертає true, якщо досягнення було додано, і false, якщо таке досягнення вже існує.

9. Колекція Map<String, String>

У класі Main додатково використано колекцію Map<String, String>.

Вона використовується для демонстрації асоціативної структури ключ-значення.

```
Map<String, String> collectionsInfo = Map.of(  
    "ArrayList<GameItem>", "зберігає предмети у порядку додавання",  
    "LinkedHashSet<Achievement>", "зберігає унікальні досягнення без  
    дублікатів"  
);
```

Обхід Map виконується через entrySet():

```
for (Map.Entry<String, String> entry : collectionsInfo.entrySet()) {  
    System.out.println(entry.getKey() + " — " + entry.getValue());  
}
```

Це показує роботу з парами ключ-значення.

10. UML-діаграма

Для моделі було створено UML-діаграму. На діаграмі показано класи предметної області, сервісний клас та використання стандартних колекцій Java.

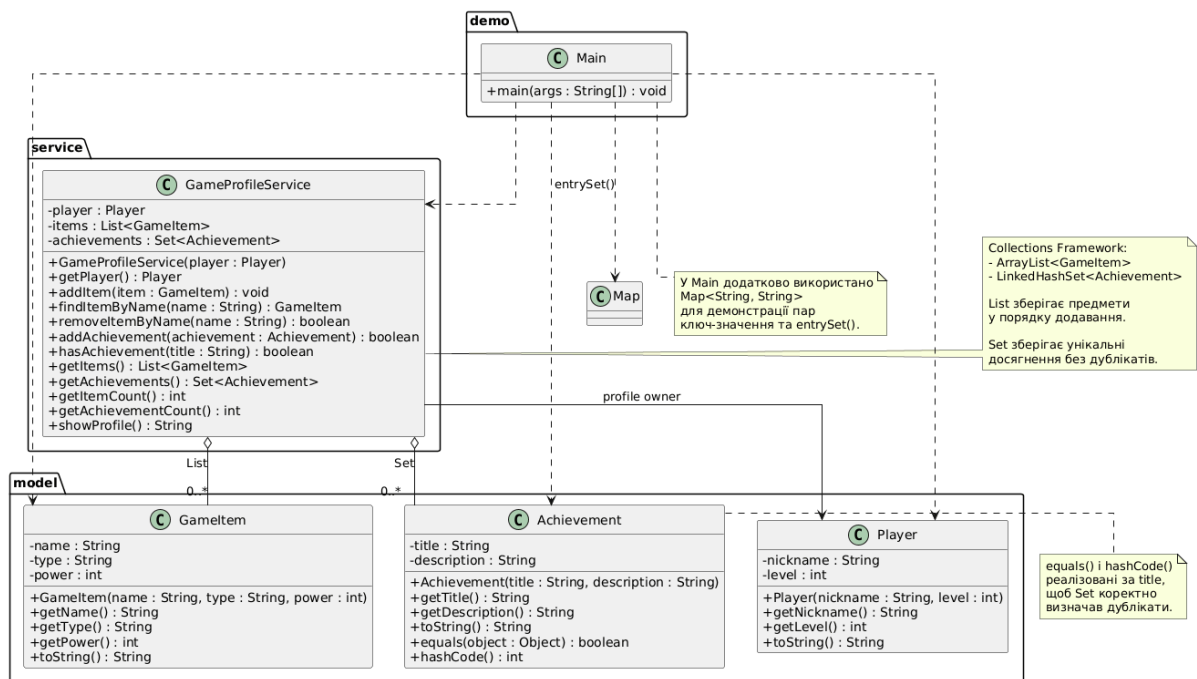


Рисунок 1 — UML-діаграма моделі ігрового профілю з використанням стандартних колекцій Java

11. Основні операції програми

У програмі реалізовано такі операції:

1. Створення гравця.
2. Додавання ігрових предметів.
3. Пошук предмета за назвою.
4. Видалення предмета за назвою.
5. Додавання досягнення.
6. Перевірка наявності досягнення.
7. Заборона дублювання однакових досягнень.
8. Перегляд усіх предметів.
9. Перегляд усіх досягнень.
10. Формування текстового опису профілю.

Ці операції демонструють практичне використання стандартних колекцій Java.

12. Демонстраційна програма

У класі Main виконується демонстрація роботи програми.

Послідовність дій:

1. Створюється гравець.
2. Створюється сервіс GameProfileService.
3. Створюються ігрові предмети.
4. Предмети додаються до профілю.
5. Додаються досягнення.
6. Виконується спроба додати дублікат досягнення.
7. Виводиться повний профіль гравця.
8. Виконується пошук предмета.
9. Виконується видалення предмета.
10. Виводяться предмети після видалення.
11. Виводяться досягнення.
12. Демонструється використання Map і entrySet().

```
Ігровий профіль
Гравець: Artem, рівень: 15

Предмети:
- Предмет: Сталевий меч, тип: Зброя, сила: 25
- Предмет: Залізна броня, тип: Броня, сила: 18
- Предмет: Зілля лікування, тип: Витратний предмет, сила: 10

Досягнення:
- Досягнення: Перший бій – Гравець переміг першого ворога
- Досягнення: Колекціонер – Гравець отримав три предмети

Кількість предметів: 3
Кількість досягнень: 2

Пошук предмета:
Предмет: Сталевий меч, тип: Зброя, сила: 25

Видалення предмета:
Предмет видалено

Предмети після видалення:
Предмет: Сталевий меч, тип: Зброя, сила: 25
Предмет: Залізна броня, тип: Броня, сила: 18

Досягнення через for-each:
Досягнення: Перший бій – Гравець переміг першого ворога
Досягнення: Колекціонер – Гравець отримав три предмети

Process finished with exit code 0
```

Рисунок 2 — результат запуску програми

13. Тестування

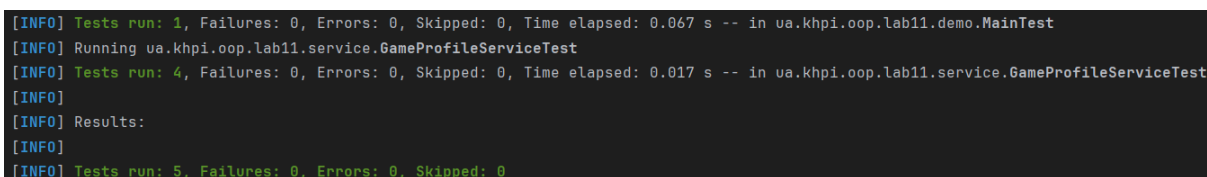
Для перевірки роботи програми були створені тести JUnit Jupiter.

Тестові класи:

- GameProfileServiceTest;
- MainTest.

Тести перевіряють:

- додавання предметів;
- пошук предмета за назвою;
- видалення предмета;
- зміну кількості предметів після видалення;
- додавання досягнень;
- заборону дублювання досягнень через Set;
- перевірку наявності досягнення;
- формування тексту профілю;
- запуск класу Main без помилок.



```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.067 s -- in ua.khpi.oop.lab11.demo.MainTest
[INFO] Running ua.khpi.oop.lab11.service.GameProfileServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.017 s -- in ua.khpi.oop.lab11.service.GameProfileServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
```

Рисунок 3 — результат виконання тестів

14. Приклади тестування

14.1 Тестування додавання і пошуку

У тестах перевіряється, що після додавання предметів сервіс може знайти їх за назвою.

```
service.addItem(sword);
```

```
service.addItem(armor);
```

```
assertEquals(2, service.getItemCount());
```

```
assertEquals(sword, service.findItemByName("Сталевий меч"));
```

```
assertEquals(armor, service.findItemByName("Залізна броня"));
```

Також перевіряється ситуація, коли предмет не знайдено:

```
assertNull(service.findItemByName("Невідомий предмет"));
```

14.2 Тестування видалення

У тестах перевіряється видалення предмета за назвою.

```
boolean removed = service.removeItemByName("Зілля лікування");
```

```
assertTrue(removed);
```

```
assertEquals(1, service.getItemCount());
```

```
assertNull(service.findItemByName("Зілля лікування"));
```

Це підтверджує, що елемент видаляється зі списку, а внутрішній стан сервісу залишається коректним.

14.3 Тестування унікальності досягнень

У тестах перевіряється, що однакові досягнення не додаються повторно.

```
boolean addedFirst = service.addAchievement(first);
```

```
boolean addedDuplicate = service.addAchievement(duplicate);
```

```
assertTrue(addedFirst);
```

```
assertFalse(addedDuplicate);
```

```
assertEquals(1, service.getAchievementCount());
```

Це підтверджує правильність використання Set.

15. Обґрунтування вибору колекцій

Для предметів було обрано ArrayList, тому що важливо зберігати порядок додавання та зручно виконувати послідовний перегляд.

Для досягнень було обрано LinkedHashSet, тому що досягнення мають бути унікальними, але при цьому бажано зберігати порядок їх додавання.

Для демонстрації асоціативного зберігання використано Map, яка дозволяє зберігати дані у вигляді пари ключ-значення.

Таким чином, кожна колекція має окрему роль у програмі.

16. Висновок

У ході виконання лабораторної роботи було реалізовано консольну Java-програму для роботи з ігровим профілем.

Було створено класи Player, GameItem, Achievement та сервіс GameProfileService.

У роботі використано стандартні колекції Java: ArrayList для зберігання предметів, LinkedHashSet для зберігання унікальних досягнень і Map для демонстрації асоціативної структури ключ-значення.

Було реалізовано операції додавання, пошуку, перегляду та видалення даних.

Тести підтвердили коректність роботи сервісу: предмети додаються, знаходяться та видаляються, а дублікати досягнень не додаються завдяки використанню Set.

Отже, лабораторна робота показала, як використовувати Collections Framework у Java та як правильно обирати колекції відповідно до вимог задачі.